

选择题

1. (B) 系统体系结构的最佳表示形式是一个可执行的软件原型。

A. 真
B. 假

软件体系结构通常通过模型（如UML图、ADL描述）来表示，强调的是系统的高层结构、组件和交互。原型主要用于验证需求或降低技术风险，而不是体系结构的最佳表示形式。

2. (A) 软件体系结构描述是不同项目相关人员之间进行沟通的使能器。

A. 真
B. 假

软件体系结构描述作为沟通桥梁，帮助客户、开发人员、管理者等利益相关者理解系统结构。

3. (A) 良好的分层体系结构有利于系统的扩展与维护。

A. 真
B. 假

分层架构（Layered Architecture）实现了关注点分离，使得某的一层的修改（如更换数据库或UI）不会影响到其他层，提高了可维护性和可扩展性。

4. (B) 消除两个包之间出现的循环依赖在技术上是不可行的。

A. 真
B. 假

循环依赖（Cyclic Dependency）的消除在技术上完全可行且应该被消除。可以通过引入新的公共包、使用接口回调（依赖倒置原则）等方法来解耦。

5. (A) 设计模式是从大量成功实践中总结出来且被广泛公认的实践和知识。

A. 真
B. 假

6. 程序编译器的体系结构适合使用 (A) 。

A. 仓库体系结构
B. 模型—视图—控制器结构
C. 客户机 / 服务器结构
D. 以上选项都不是

编译器通常使用仓库风格（或黑板风格），其中包含一个中心数据结构（符号表、语法树），词法分析、语法分析等组件均围绕该中心数据进行交互。

7. 网站系统是一个典型的（ C ）。

- A. 仓库体系结构
- B. 胖客户机 / 服务器结构
- C. 瘦客户机 / 服务器结构
- D. 以上选项都不是

网站系统中，客户端（浏览器）主要负责展示，业务逻辑主要在服务器端，因此属于瘦客户机模式。

8. 在分层体系结构中，（ C ）实现与实体对象相关的业务逻辑。

- A. 表示层
- B. 持久层
- C. 实体层
- D. 控制层

在标准的领域驱动设计或面向对象设计中，实体层（Entity/Domain Layer）包含具有业务逻辑的领域对象。

选择题

1. (**A**) 面向对象设计是在分析模型的基础上, 运用面向对象技术生成软件实现环境下的设计模型。

A. 真
B. 假

面向对象设计 (OOD) 是将面向对象分析 (OOA) 的结果转化为特定实现环境下的解决方案的过程。

2. (**A**) 系统设计的主要任务是细化分析模型, 最终形成系统的设计模型。

A. 真
B. 假

3. (**B**) 关系数据库可以完全支持面向对象的概念, 面向对象设计中的类可以直接对应到关系数据库中的表。

A. 真
B. 假

存在对象-关系阻抗不匹配 (Object-Relational Impedance Mismatch)。关系数据库不支持继承、多态等OO特性, 类与表通常需要通过ORM (对象关系映射) 进行转换, 不能简单地说“完全支持”或“直接对应”。

4. (**A**) 用户界面设计对于一个系统的成功是至关重要的, 一个设计得很差的用户界面可能导致用户拒绝使用该系统。

A. 真
B. 假

可用性是软件质量的重要属性。

5. 内聚表示一个模块 (**B**) 的程度, 耦合表示一个模块 (**D**) 的程度。

A. 可以被更加细化
B. 仅关注在一件事情上
C. 能够适时地完成其功能
D. 联接其他模块和外部世界

内聚 (Cohesion): 模块内部各元素彼此结合的紧密程度, 即 B. 仅关注在一件事情上 (功能内聚)。

耦合 (Coupling): 模块之间互连的程度, 即 D. 联接其他模块和外部世界。

6. 良好设计的特征是 (**E**)。

- A. 模块之间呈现高耦合
- B. 实现分析模型中的所有需求
- C. 包括所有组件的测试用例
- D. 提供软件的完整描述
- E. 选项 B 和 D
- F. 选项 B、C 和 D

A. 模块间高耦合（错误，应为低耦合）。

B. 实现分析模型中的所有需求（正确，完整性/可追溯性）。

C. 包括所有组件的测试用例（通常测试用例属于测试文档，不直接包含在设计模型中）。

D. 提供软件的完整描述（正确）。

7. （ A ）是选择合适的解决方案策略，并将系统划分成若干子系统，从而建立整个系统的体系结构；（ B ）细化原有的分析对象，确定一些新的对象、对每一个子系统接口和类进行准确详细的说明。

- A. 系统设计
- B. 对象设计
- C. 数据库设计
- D. 用户界面设计

系统设计（System Design）：高层设计，划分于系统、定义架构。

对象设计（Object Design）：详细设计，定义类、接口和具体逻辑。

8. 下面的（ D ）界面设计原则不允许用户保持对计算机交互的控制。

- A. 允许交互中断
- B. 允许交互操作取消
- C. 对临时用户隐藏技术内部信息
- D. 只提供一种规定的方法完成任务

用户界面设计原则强调“用户控制权”（User Control）。

A（允许中断）、B（允许取消）都给予了用户控制权。

C（隐藏内部信息）是减轻用户负担，与控制权不冲突。

D 强制用户只能按一种方式操作，剥夺了用户的灵活性和控制权，是违反允许用户保持控制这一原则的。

作业

- 下图是某公司支付雇员薪水程序的一个简化 UML 设计类图，目前雇员薪水是按固定月薪支付的，系统需要准时支付正确的薪金，并从中扣除各种扣款。现在该公司准备增加“时薪”和“底薪+佣金”两种支付方式，考虑到良好的可扩展性，开发人员打算使用设计模式修改原有设计，以支持多种薪水支付方式。

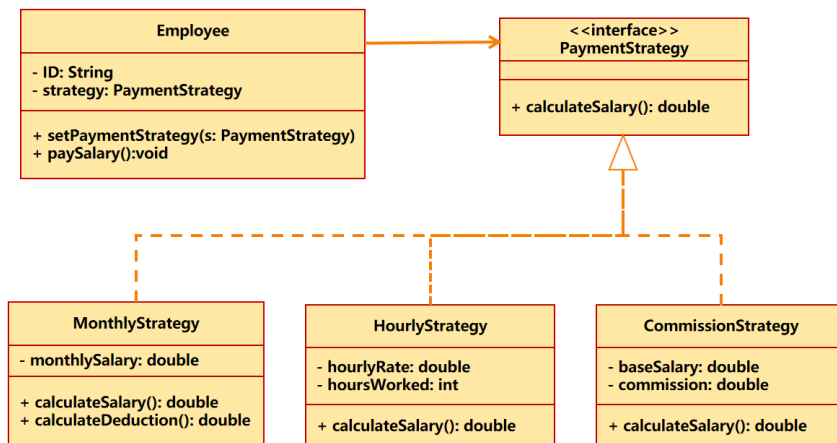
(1) 你会选择什么设计模式？为什么？

我会选择**策略模式**。

理由：原有的系统只支持“固定月薪”，但新的需求引入了“时薪”和“底薪+佣金”等多种计算方式。薪水的计算逻辑是一个经常变化的“算法族”。策略模式可以将不同的计算逻辑（固定月薪、时薪、佣金）封装在独立的策略类中。当需要增加新的支付方式时，只需增加一个新的策略类，而无需修改原有的Employee类或其他代码，易于扩展。这使得算法的变化独立于使用算法的客户（Employee），符合开放封闭原则（OCP）。同时，策略模式满足多态替换，可以在运行时动态地改变雇员的薪水支付方式。

(2) 请画出修改后的 UML 设计类图，并用C++语言编写实现该类图的程序。

引入抽象接口**PaymentStrategy**，将每种具体的薪资计算逻辑**封装在独立的类中**（如MonthlyStrategy, HourlyStrategy, CommissionStrategy）。Employee类不再直接包含计算逻辑，而是**持有一个PaymentStrategy类型的引用**。通过**setPaymentStrategy**方法，可以在运行时**灵活地改变雇员的薪资计算方式**。



C++程序如下：

```
#include <iostream>
#include <string>

using namespace std;
```

```

// -----
// 1. 策略接口 (Strategy Interface)
// 对应 UML 中的 <<interface>> PaymentStrategy
// -----
class PaymentStrategy {
public:
    // 虚析构函数，确保子类析构时正确释放资源
    virtual ~PaymentStrategy() {}

    // 纯虚函数，对应 UML 中的 calculateSalary(): double
    virtual double calculateSalary() = 0;
};

// -----
// 2. 具体策略类 (Concrete Strategies)
// -----

// A. 固定月薪策略
class MonthlyStrategy : public PaymentStrategy {
private:
    double monthlySalary;

public:
    MonthlyStrategy(double salary) : monthlySalary(salary) {}

    // 假设扣除 10% 的税
    double calculateDeduction() {
        return monthlySalary * 0.1;
    }

    double calculateSalary() override {
        // 薪水 = 固定月薪 - 扣款
        return monthlySalary - calculateDeduction();
    }
};

// B. 时薪策略
class HourlyStrategy : public PaymentStrategy {
private:
    double hourlyRate;
    int hoursWorked;

public:
    HourlyStrategy(double rate, int hours) : hourlyRate(rate),
hoursWorked(hours) {}

    double calculateSalary() override {
        return hourlyRate * hoursWorked;
    }
};

```

```

    }
};

// C. 底薪 + 佣金策略
class CommissionStrategy : public PaymentStrategy {
private:
    double baseSalary;
    double commission;

public:
    CommissionStrategy(double base, double comm) : baseSalary(base),
    commission(comm) {}

    double calculateSalary() override {
        return baseSalary + commission;
    }
};

// -----
// 3. 环境类 (Context)
// 对应 UML 中的 Employee
// -----
class Employee {
private:
    string ID;
    PaymentStrategy* strategy; // 聚合关系: Employee 持有 Strategy 指针

public:
    // 构造函数
    Employee(string id) : ID(id), strategy(nullptr) {}

    // 注入或更换策略
    void setPaymentStrategy(PaymentStrategy* s) {
        this->strategy = s;
    }

    // 执行薪水支付
    void paySalary() {
        if (strategy == nullptr) {
            cout << "错误: 员工 " << ID << " 未设置薪资策略。" << endl;
            return;
        }

        double amount = strategy->calculateSalary();
        cout << "向员工 [ID: " << ID << "] 支付薪水: " << amount << " 元"
        << endl;
    }
};

```

